RT-IoT2022 Multi-Class IoT Attack Detection Project

Instruction Manual

Faculty: Dr. Jilan Samiuddin
Teaching Assistant: Bijoy Deb Babu
Course: CSE 445 (Machine Learning), Section 8

July 13, 2026

Abstract

This document describes the RT-IoT2022 Multi-Class IoT Attack Detection Project. Students will build a complete machine learning pipeline using a tabular IoT network traffic dataset to classify network traffic into different attack/traffic categories. This is a multi-class classification problem with 12 target classes. Students must perform data cleaning, exploratory data analysis, preprocessing, model training, validation, testing, result visualization, and model limitation analysis. Final grading is based on the notebook (50 marks) and the written report (40 marks). There will be a presentation (10 marks) at the end of the semester, where each group will be required to present and defend their work.

1 Project Overview

In this project, students will work with the **RT-IoT2022** dataset, a real-time Internet of Things (IoT) network traffic dataset. The goal is to build machine learning models that can identify the type of IoT network traffic or cyberattack based on tabular network features.

Students must build a full machine learning workflow, including data loading, data cleaning, exploratory data analysis (EDA), preprocessing, train/validation/test splitting, model training, model tuning if applicable, evaluation, result visualization, and final discussion.

- **Project Type:** Multi-class classification
- **Dataset:** RT-IoT2022
- **Domain:** IoT Cybersecurity / Network Intrusion Detection
- **Target:** Attack/traffic class column in the dataset
- **Number of Classes:** 12
- **Main Task:** Predict the correct network traffic or attack class

2 Dataset Information

The RT-IoT2022 dataset is publicly available from the UCI Machine Learning Repository. It contains network traffic data collected from a real-time IoT infrastructure. The dataset includes both normal and attack traffic patterns from IoT network environments.

Property	Description
Dataset Name	RT-IoT2022
Source	UCI Machine Learning Repository
Dataset Type	Tabular / Multivariate / Sequential
Number of Instances	123,117
Number of Features	84
Number of Target Classes	12
Task	Multi-class IoT attack classification
Domain	Cybersecurity / IoT Network Traffic

The dataset can be accessed from the following official link:

RT-IoT2022 Dataset – UCI Machine Learning Repository

Students may download the dataset manually from the UCI page or load it using the official UCI Python package. The dataset must be used as a multi-class classification dataset. Students are not allowed to remove minority classes only to improve performance.

3 Project Objectives

By completing this project, students will be able to:

- Understand the structure of a real-world IoT network intrusion detection dataset.
- Identify the target variable and separate it correctly from input features.
- Perform exploratory data analysis to understand class distribution, feature behavior, missing values, and possible outliers.
- Detect class imbalance and explain its effect on model performance.
- Apply proper preprocessing steps such as encoding, scaling, and data splitting.
- Avoid data leakage by fitting preprocessing tools only on training data.
- Train and evaluate multiple machine learning models for multi-class classification.
- Compare models using several metrics such as Accuracy, Precision, Recall, F1-score, Macro F1-score, and Weighted F1-score.
- Visualize metric changes using graphs where applicable, especially for models trained over epochs.
- Analyze confusion matrices and identify which classes are most frequently misclassified.
- Discuss the limitations of the trained models.
- Write a clear, structured, and reproducible technical report.

4 Expected Deliverables

Students must submit the following files:

- **Source Python Notebook** (`.ipynb`) containing all required code, outputs, plots, tables, metrics, and confusion matrices.
- **Report** in PDF format (`.pdf`), 5–12 pages including references and appendix if needed. Doc format is not allowed.
- **Submission Format:** Upload both Python notebook & report files together in a single `.zip` file. **`.rar` files are not allowed.**

5 Minimum Project Requirements

Each student/group must complete the following:

- Load the RT-IoT2022 dataset properly.
- Identify and separate the feature columns and target column.
- Perform data cleaning and exploratory data analysis.
- Apply suitable preprocessing methods.
- Split the dataset into proper training, validation, and testing sets if model tuning is performed.
- Train at least **three** different machine learning models.
- Evaluate all models using multiple evaluation metrics.

- Select the best model using balanced reasoning based on all major metrics, not only one metric.
- Include metric comparison tables and graphs in the notebook and report.
- Discuss the limitations of the final model.

6 Recommended Data Split

Students must use a fair and reproducible data split. The recommended split is:

- **Training Set:** 70%
- **Validation Set:** 15%
- **Testing Set:** 15%

The validation set should be used for model tuning, hyperparameter selection, and model comparison during development. The test set must be used only for final evaluation. The test data must be used only for testing, not for tuning or any other work. If students do not perform model tuning, they may use an 80/20 train/test split. However, students can choose their own splits for better results.

7 Models

Students must train at least **three** different classification models from the models taught in the course. Variations of the models are allowed as long the main backbone is one of those taught in the course.

8 Required Evaluation Metrics

Since this is a multi-class classification problem and the dataset may contain class imbalance, students must not depend on only one metric. Accuracy, macro precision, macro recall, and macro F1-score should all be considered.

Students must report the following metrics for every model:

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1-score
- Weighted Precision
- Weighted Recall
- Weighted F1-score
- Confusion Matrix
- Classification Report

Students should explain the meaning of the important metrics in the context of this dataset. For example, high accuracy alone may not be enough if the model performs poorly on minority attack classes.

9 Metric Visualization Requirements

Students must clearly visualize and report model performance using graphs.

- Show a comparison graph of all trained models using major metrics such as Accuracy, Precision, Recall, Macro F1-score, and Weighted F1-score.
- Show confusion matrix heatmaps for all trained models or at least for the main models used in final comparison.
- For models trained over epochs, such as MLP or neural network-based models, show metric changes per epoch.
- Epoch-wise graphs should include training and validation curves where applicable.
- Recommended epoch-wise graphs include:
  - Training loss vs validation loss
  - Training accuracy vs validation accuracy
  - Training precision vs validation precision
  - Training recall vs validation recall
  - Training F1-score vs validation F1-score
- Every graph must have a clear title, axis labels, and short explanation.

If a model does not train over epochs, such as Random Forest or Logistic Regression, students do not need to show per-epoch graphs for that model. However, final metric comparison graphs are still required.

10 Notebook Marking Rubric (Total: 50 Marks)

The notebook will be graded using the following criteria:

1. **Dataset Loading & Understanding** **(4 marks)**
   - Load the dataset correctly.
   - Show the dataset shape, column names, and basic information.
   - Correctly identify the target column and feature columns.
   - Show the number of target classes.
2. **Data Cleaning** **(4 marks)**
   - Show missing value counts per column and total missing values.
   - Check and report duplicate rows.
   - Handle missing values, duplicates, or invalid values using justified methods.
   - Show evidence of the final cleaned dataset.
3. **Exploratory Data Analysis (EDA)** **(10 marks)**
   - Show the class distribution of the target variable.
   - Discuss whether the dataset is balanced or imbalanced.
   - Show at least three meaningful feature distribution plots.
   - Include a correlation heatmap for selected numerical features.
   - Explain important observations from EDA using short Markdown descriptions.
   - Identify any meaningful outliers or unusual patterns if present.
4. **Preprocessing & Data Splitting** **(8 marks)**
   - Perform proper train/validation/test split if tuning is done.
   - Use the test set only for final evaluation.

- Apply encoding for categorical features if needed.
- Apply feature scaling where appropriate.
- Ensure no data leakage: fit encoders and scalers only on training data.
- Keep the preprocessing implementation clean and reproducible.

5. **Model Training** **(10 marks)**
   - Train at least **three** different classification models.
   - Use the same data split for all models.
   - Use suitable model parameters and random seeds where applicable.
   - Clearly show training and prediction steps.
   - Present model results in a clean comparison table.
6. **Model Evaluation & Visualization** **(8 marks)**
   - Report Accuracy, Precision, Recall, Macro F1-score, and Weighted F1-score for each model.
   - Show final metric comparison graphs for the trained models.
   - Show confusion matrix heatmaps.
   - For epoch-based models, show training and validation metric curves per epoch.
   - Explain the graphs clearly using short Markdown descriptions.
7. **Result Summary & Best Model Selection** **(6 marks)**
   - Summarize the final results in a clear table.
   - Select the best model using balanced reasoning based on several metrics.
   - Highlight which model performed best in Accuracy, Precision, Recall, Macro F1-score, and Weighted F1-score.
   - Show the classification report of the selected best model.
   - Clearly mention the final selected model in the notebook.

11 Report Marking Rubric (Total: 40 Marks)

The written report will be graded using the following criteria:

1. **Problem Statement & Dataset Description** **(8 marks)**
   Clearly describe the project problem, dataset source, dataset size, target variable, number of features, and number of classes.
2. **EDA, Cleaning & Preprocessing Explanation** **(8 marks)**
   Present the important EDA findings, data cleaning steps, preprocessing decisions, and data split strategy with proper explanations.
3. **Modeling Methodology** **(8 marks)**
   Explain the selected models, training approach, validation process, and tuning strategy if any tuning was performed.
4. **Results, Graphs & Evaluation** **(8 marks)**
   Present final results using metric tables, comparison graphs, confusion matrices, classification reports, and epoch-wise graphs where applicable.
5. **Discussion, Best Model & Limitations** **(8 marks)**
   Discuss the best model using multiple metrics, explain class-wise performance, identify common misclassifications, and mention model limitations.

12 Grading Scheme (Total: 100 Marks)

Your final grade for this project will be calculated as follows:

Component	Marks
Notebook submission (`.ipynb`)	50
Report submission (`.pdf`)	40
Presentation	10
Total	**100**

13 Report Writing Guidelines

The report must be written in a proper structured format. Students **do not** need to follow any traditional IEEE, ACM, or journal template. However, the report must be organized clearly with separate sections for each major part of the project.

The report should clearly explain:

- What was done in each step.
- Why each step was necessary.
- What was found from the EDA.
- How the data was cleaned and preprocessed.
- How the models were trained and evaluated.
- What the metric tables and graphs show.
- Which model performed best and why.
- What limitations exist in the final model.
- The report must be submitted in PDF format.

All important EDA results shown in the notebook must be included in the report with proper descriptions. Students must not only paste figures or tables. Every important figure, table, metric, and graph must be explained in words.

14 Academic Integrity and General Rules

- **Reproducibility:** Keep your notebook runnable end-to-end. The notebook will be executed in Google Colab to check whether it can produce the reported outputs.
- **No Data Leakage:** Perform data splitting before fitting encoders, scalers, or any preprocessing tools.
- **Fair Modeling:** Do not remove minority classes or modify target labels unfairly to improve performance.
- **Clear Explanation:** Use clear headings and short Markdown explanations in the notebook.
- **Plagiarism Strictly Prohibited:** Copying notebooks, reports, code, explanations, or results from classmates, online sources, GitHub, Kaggle, blogs, or AI tools without proper understanding and citation is strictly prohibited.
- **External Resources:** Any external code, article, library documentation, or online resource used must be cited in the report.
- **Original Work:** Students must write their own code, produce their own results, and explain their own findings.
- **Submission Format:** Submit the notebook and report together in a single `.zip` file. `.rar` files are not allowed.